



Идентитет, audit, observability и конфигурација

Попречни слој за разумевање тока

Тема и проблем

Разумети како се операција прати кроз више компоненти без губитка контекста.

Када се ток прекине, засебни логови не дају одговор ко је покренуо радњу, шта се догодило и под којим условима.

НАПОМЕНА

Заједнички контекст и audit омогућавају реконструкцију тока, док конфигурација чува видљиве услове под којима систем ради.

Четири појма теме

Дистрибуирани систем мора моћи да одговори на четири питања: ко је покренуо радњу, шта се догодило, кроз које компоненте је ток прошао и под којим условима је радио.

01

Идентитет

ко покреће радњу

02

Корелација

један ток кроз компоненте

03

Audit

пословно значајна одлука

04

Конфигурација

услови рада система

Идентитет

Идентитет показује ко или шта је покренуло радњу. Без њега није могуће повезати захтев са одговорношћу и правом приступа.

- 01** Повезује захтев са одговорношћу и правом приступа.
- 02** Проверава се и улога и припадност мисији.
- 03** Нема одлуку о приступу без познатог идентитета.

Корелација

Исти корелациони идентификатор прати операцију кроз API, ред и worker. Он омогућава да се више техничких трагова прочита као један ток.

- 01** Исти идентификатор кроз API, ред и worker.
- 02** Спаја техничке трагове у један разумљив ток.
- 03** Не сме се регенерисати у сваком слоју.

Audit

Audit бележи одлуке које имају пословни или безбедносни значај: ко је шта променио, када и са којим исходом. Није замена за сваки технички лог.

- 01 Бележи пословно и безбедносно значајне одлуке.
- 02 Append-only и претражив по correlation id-y.
- 03 Није замена за сваки технички лог.

Конфигурација

Конфигурација одређује под којим условима компонента ради. Валидне вредности, извор и промена конфигурације морају бити видљиви тиму.

- 01** Одређује услове рада компоненте.
- 02** Валидан опсег и извор морају бити видљиви.
- 03** Промена конфигурације је следљива, не тиха.

Пример: захтев пролази кроз три компоненте

Корисник покреће операцију кроз API, API објављује поруку, а worker завршава обраду.

- 01 Корелациони идентификатор настаје на граници тока и преноси се даље.
- 02 Audit чува одлуке које утичу на одговорност, док лог чува технички контекст.
- 03 Конфигурација одређује очекивано понашање, као што су рокови и прагови, и мора бити проверљива.

Шема следљивог дистрибуираног тока

Један контекст пролази кроз све компоненте, па се неуспех може објаснити без нагађања.

Елемент	Улога у току
Улазни захтев	идентитет и корелација
API	прихвата или одбија
Порука и worker	обрађују исти контекст
Audit и траг	објашњавају исход

Покретање примера

Пример прати једну операцију кроз две компоненте.



```
1 cd examples/odp-dotnet/vezba-03-identitet-audit-konfiguracija
2 dotnet run --project src/Odp.Vezba03.ConsoleUi
3 dotnet test Odp.Vezba03.sln
```

Правило у коду

Исти CorrelationId види се у логу станице и у ревизијском трагу центра.

```
1 public ScheduleContactResult Schedule(ContactRequest request, OperationContext context)
2 {
3     ArgumentNullException.ThrowIfNull(request);
4     ArgumentNullException.ThrowIfNull(context);
5
6     var code = Decide(request, context);
7
8     auditLog.Record(new AuditEntry(
9         clock.UtcNow,
10        context.CorrelationId,
11        context.ActorId,
12        Action,
13        request.StationId,
14        code));
15
16     return new ScheduleContactResult(code == ContactCodes.Scheduled, code, context.CorrelationId);
17 }
```

Тест који задаје квар

Ревизијски запис настаје и када је захтев одбијен пре позива станице.

```
1 [Test]
2 public void Schedule_CarriesTheSameCorrelationIdToTheStationAndTheAuditTrail()
3 {
4     var result = _handler.Schedule(Request, Context);
5
6     using (Assert.EnterMultipleScope())
7     {
8         Assert.That(result, Is.EqualTo(new ScheduleContactResult(true, ContactCodes.Scheduled, "op-42")));
9         Assert.That(_stationGateway.Calls.Single().CorrelationId, Is.EqualTo("op-42"));
10        Assert.That(
11            _auditLog.Entries.Single(),
12            Is.EqualTo(new AuditEntry(
13                Now,
14                "op-42",
15                "ana",
16                "contact.schedule",
17                "GS-NOVI-SAD",
18                ContactCodes.Scheduled)));
19    }
20 }
```

Испис демонстрације

Неисправна конфигурација зауставља покретање са кодом грешке.

```
1 op-0001: GS-NOVI-SAD, 10 min -> contact_scheduled
2 op-0002: GS-NOVI-SAD, 40 min -> duration_too_long
3 op-0003: GS-BEOGRAD, 10 min -> station_unavailable
4
5 Log stanice:
6   op-0001 | GS-NOVI-SAD | reserved=True
7   op-0003 | GS-BEOGRAD | reserved=False
8
9 Revizijski trag:
10  op-0001 | ana | contact.schedule | GS-NOVI-SAD | contact_scheduled
11  op-0002 | marko | contact.schedule | GS-NOVI-SAD | duration_too_long
12  op-0003 | jelena | contact.schedule | GS-BEOGRAD | station_unavailable
```

Принципи и замке

Принцип важи тек када је препозната и замка која га тихо поништава.

ПРИНЦИПИ

Audit чува одлуке које су важне за одговорност и истрагу.

Логови и трагови чувају технички контекст, али не тајне.

Конфигурација има валидно значење, власника и начин промене.

ЗАМКЕ

Лог који открива креденцијале или личне податке.

Корелациони идентификатор креиран у свакој компоненти уместо пренет.

Конфигурација прихваћена без провере опсега и утицаја.

Типичне грешке

Ове грешке се најчешће понављају када се принцип површно примени.

- 01** Тајне у логовима.
- 02** Нови корелациони идентификатор у сваком слоју.
- 03** Конфигурација без валидног опсега.

Како се принцип примењује

Кораци показују како се принцип препознаје и доследно спроводи у развоју.

- 01** Пратити неколико важних токова од захтева до исхода.
- 02** Преносити исти контекст кроз границе.
- 03** Тестирати промењен или невалидан услов рада.

Питање за дискусију

Одговор се образлаже failure сценаријем и тестом, а не описом срећног пута.

Како бисте за један неуспешан захтев спојили траг из API-ја, реда порука и workerа?

ЗА ВЕЖБУ

Како бисте реконструисали једну неуспелу операцију која је прошла кроз API, ред порука и worker, а да не читате неограничену количину логова?

Задатак уз пример

Проширење примера уз вежбу; правило иде у домен, а доказ у тест.

- 01 Додати у `OperationContext` идентификатор позивајуће компоненте (`Source`) и уписати га у ревизијски запис.
- 02 Проширити тест тако да доказује да запис садржи и извор операције.

Примена на пројекат

Принцип са вежбе примењује се на додељену пројектну целину.

- 01** Идентификовати конкретну дистрибуирану одлуку у својој целини.
- 02** Имплементирати је са стабилним кодом исхода.
- 03** Припремити поновљив failure сценарио и тест који га доказује.

Резиме

Тим бира неколико операција које мора умети да прати од захтева до исхода.

Један контекст пролази кроз све компоненте, па се неуспех може објаснити без нагађања.

НАПОМЕНА

Наредна вежба: Верзионисање и failure-first тестирање.